

O&G · 6 — Discovering the decline law itself from production data ⇌

Every earlier notebook in this series *assumed* a model form (Arps, a Fetkovich aquifer, the diffusivity equation) and either fit its parameters or filled a gap with a neural network. This closing notebook asks a more basic question: given nothing but a rate-vs-time history, can we discover the governing decline **equation** itself, with no assumed functional form at all?

We use **SINDy** ([06_sindy_model_discovery.jl](#)) on a well whose true (but, to the algorithm, unknown) behavior is harmonic decline ($b = 1$ in the Arps family), which happens to correspond to the clean autonomous ODE $\dot{q} = -\frac{D_i}{q_i} q^2$ — a single quadratic term. If SINDy is given a library rich enough to contain that term, sparse regression should isolate it automatically.

```
1 using DataDrivenDiffEq, DataDrivenSparse, ModelingToolkit, OrdinaryDiffEq, Random, Plots
```

1. A harmonically declining well ⇌

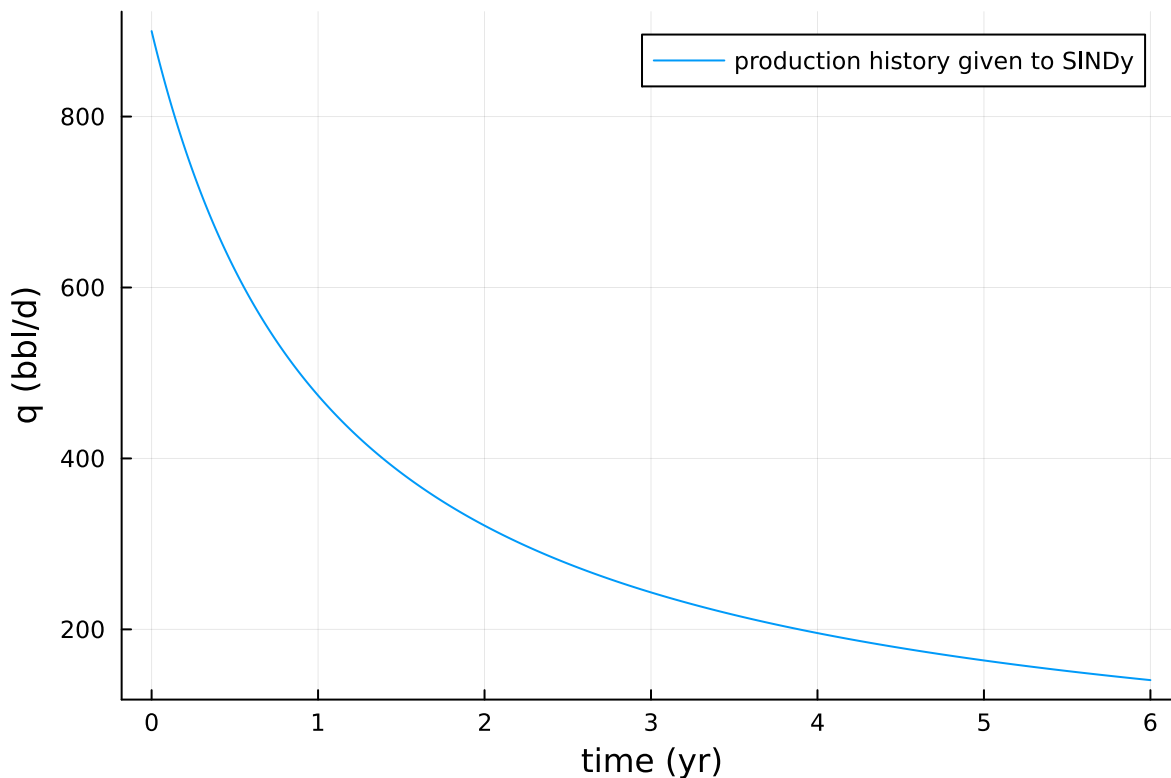
$$\dot{q} = -\frac{D_i}{q_i} q^2$$

harmonic_decline! (generic function with 1 method)

```
1 function harmonic_decline!(du, u, p, t)
2     Di, qi = p
3     q = u[1]
4     du[1] = -Di * q^2 / qi
5     return nothing
6 end
```

► [0.0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, (

```
1 begin
2   rng_og6 = Random.default_rng()
3   Random.seed!(rng_og6, 9)
4
5   qi_og6, Di_og6 = 900.0, 0.9 # bbl/d, 1/yr
6   tspan_og6 = (0.0, 6.0)
7   saveat_og6 = 0.01
8
9   well_prob = ODEProblem(harmonic_decline!, [qi_og6], tspan_og6, (Di_og6, qi_og6))
10  well_sol = solve(well_prob, Tsit5(); saveat=saveat_og6)
11
12  Xq = Array(well_sol)
13  tq = well_sol.t
14 end
```



```
1 plot(tq, Xq[1, :]; xlabel="time (yr)", ylabel="q (bbl/d)", label="production history given to SINDy")
```

2. Build the DataDrivenProblem ⇄

As in the general SINDy notebook, we supply the derivative directly here (computable because we control the synthetic example); in practice DataDrivenDiffEq.jl can also estimate it from the trajectory by finite differencing or collocation.

```
Continuous DataDrivenProblem{Float64} ##DDProblem#296 in 1 dimensions and 601 samples
```

```
1 begin
2     DXq = similar(Xq)
3     for (i, ui) in enumerate(eachcol(Xq))
4         harmonic_decline!(view(DXq, :, i), ui, (Di_og6, qi_og6), tq[i])
5     end
6     ddprob_q = ContinuousDataDrivenProblem(Xq, tq; DX=DXq)
7 end
```

3. A polynomial candidate library ⇄

We deliberately do *not* tell SINDy the true exponent — the library contains every monomial in q up to degree 3, and sparse regression must pick out the one that actually matters.

```
Model ##Basis#301 with 4 equations
```

```
States : q
```

```
Independent variable: t
```

```
Equations
```

```
 $\varphi_1 = 1$ 
```

```
 $\varphi_2 = q$ 
```

```
 $\varphi_3 = q^2$ 
```

```
 $\varphi_4 = q^3$ 
```

```
1 begin
2     @variables q
3     decline_basis = Basis(polynomial_basis([q], 3), [q])
4 end
```

⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run `import LaTeXify` to enable pretty-printing of symbolic expressions. This warning will only display once.

```
Model ##Basis#308 with 1 equations
```

```
States : q
```

```
Parameters : p1
```

```
Independent variable: t
```

```
Equations
```

```
Differential(t, 1)(q) = p1*(q2)
```

```
1 begin
2     sparse_opt_q = STLSQ(exp10.(-6:0.25:1))
3     sindy_res_q = solve(ddprob_q, decline_basis, sparse_opt_q)
4     sindy_decline_law = get_basis(sindy_res_q)
5 end
```

```
1 println(sindy_decline_law)
```

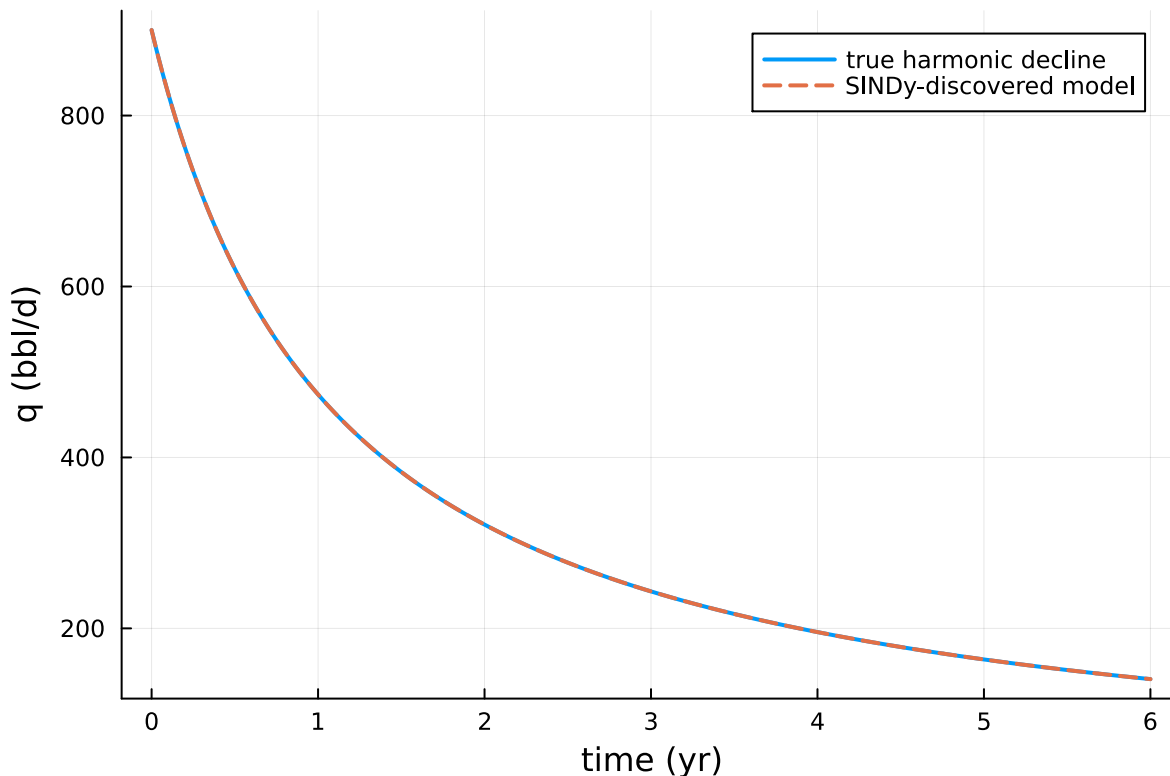
```
> Model ##Basis#308 with 1 equations
States : q
Parameters : p1
Independent variable: t
Equations
Differential(t, 1)(q) = p1*(q2)
```



► [-0.001]

```
1 get_parameter_values(sindy_decline_law)
```

4. Validate the discovered law by forecasting forward ⇄



```
1 begin
2   discovered_well_prob = ODEProblem(sindy_decline_law, [qi_og6], tspan_og6,
   get_parameter_values(sindy_decline_law))
3   discovered_well_sol = solve(discovered_well_prob, Tsit5(); saveat=saveat_og6)
4
5   plot(tq, Xq[1, :]; label="true harmonic decline", lw=2, xlabel="time (yr)",
   ylabel="q (bbl/d)")
6   plot!(tq, Array(discovered_well_sol)[1, :]; label="SINDy-discovered model", lw=2,
   ls=:dash)
7 end
```

Takeaways ⇄

- Given a rich-enough candidate library, SINDy recovered that this well's decline is governed by a single q^2 term — i.e. it independently rediscovered harmonic ($b = 1$) Arps decline directly from the rate history, without that model form ever being stated.
- A non-integer decline exponent ($0 < b < 1$, the common hyperbolic case) is not a single monomial, so a plain polynomial library would only approximate it; a production application

would extend the library with fractional powers of q or apply SINDy locally over the operating range, but the mechanics — build a library, run sparse regression, validate by simulating forward — stay identical.

- This closes the loop on the notebook series: [01_decline_curve_analysis.jl](#) assumed the Arps form and fit its parameters; this notebook shows that, at least for the clean cases, the form itself doesn't have to be assumed either.

